

Entwicklungsprojekt 2021/22

von Vincent Luciano und Luzie Ewert



Inhaltsverzeichnis

Zielhierarchie iteriert.....	Folie 3
Risikoanalyse iteriert.....	Folie 4
Risikoanalyse iteriert.....	Folie 5
Architekturdiagramm iteriert.....	Folie 6
Prozessassessment.....	Folie 7
Prozessassessment.....	Folie 8
PoC: Implementierung der Live ÖPNV Daten in das System.....	Folie 9
PoC: Jemanden finden, der die gleiche Strecke fährt durch GPS- Tracking und Filter.....	Folie 10
Code Einblicke.....	Folie 11
Code Einblicke.....	Folie 12
Code Einblicke.....	Folie 13
Code Einblicke.....	Folie 14
Code Einblicke.....	Folie 15
Code Einblicke.....	Folie 16
Fazit.....	Folie 17
Projektplan.....	Folie 18



Zielhierarchie iteriert

Strategische Ziele

- Umwelt schonen
- Weniger Schwarzfahren
- Mehr Kontakt zwischen Menschen
- Jeder Mensch kann fahren
- Dem Nutzer einfache Bedienung ermöglichen
- Weitflächiges Netzwerk
- Den ersten Use Case des Projekts funktionsfähig durchführen

Taktische Ziele

- Menschen dazu bringen die Anwendung zu nutzen
- Eine Community Bauen
- Die Exit-Kriterien der PoC's müssen erfüllt sein
- Fokus auf einen gelungenen Durchlauf, welcher den optimalen Use Case widerspiegelt

Operative Ziele

- Einen Prototyp bauen
- Einen MVP bauen
- Erste Nutzer bekommen und Werbung
- Feedback bekommen
- stetige Updates, zur Verbesserung der Usability
- PoC's und deren Exit Kriterien formulieren



Beim wiederholten Iterieren der Zielhierarchie wurden Punkte ergänzt die sich während des Projekts als neue Ziele herausstellten oder im Laufe des Projekts weiter in den Fokus rückten und entsprechend der geäußerten Kritik, mehr auf die individuellen Ziele für das Projekt eingehen würden. Somit wurden Ziele hinzugefügt, die den weiteren Vorgang innerhalb des Projekts festhalten sollten. Genauer genommen resultierten die in strategischen, taktischen und operativen Zielen festgehaltenen Ergänzungen aus dem dritten Audit heraus, da dort festgestellt wurde, dass die zuvor niedergeschrieben PoC's zu vielumfassend seien und in der noch übrig gebliebenen Zeit nicht alle erledigt werden können. Für einen Prototypen sollte daher eine einfache Simulation einer Fahrt mit den funktionierenden Funktionen dahinter am Ende programmiert sein.

Risikoanalyse iteriert

Risiken	Bewertung	Maßnahmen
passende API finden	mäßig	mehr Zeit für Recherche
API's könnten wegfallen	hoch	frühzeitig Alternativen suchen
fehlerhafte Entscheidung der Programmiersprache	mäßig	geeignete Programmiersprache für beide Teampartner finden
Netzwerk kreieren - setzt viele Nutzer voraus	mäßig	für Prototyp irrelevant - später, durch gezieltes Marketing
wir setzen auf die Solidarität der Menschen	mäßig	Anreize auch für Personen mit Ticket finden
Entstehung von spät erkannten Bugs in der Entwicklungsphase	hoch	Test Driven Development orientiert arbeiten
Ausfall einer der Teammitglieder	mäßig	Regelmäßige Updates geben, strukturiert zu arbeiten, gleichmäßige Aufgabenverteilung,
rechtliche Konflikte mit den Verkehrsunternehmen	hoch	für den Prototyp erstmal irrelevant



Die Risikoanalyse wurde ebenfalls überarbeitet, da einige neue Risiken im Laufe des Projekts auftraten deren Einfluss es abzuwägen galt und für welche sich Bewältigungsstrategien ausgedacht werden sollten. Die Risiken wurden in hohes, mäßiges und niedriges Risiko eingestuft und Maßnahmen wurden entwickelt. Zu den bereits elaborierten Risiken fügten sich vier weitere Risiken hinzu, welche sich aus dem dritten Audit und dem weiteren Fortschritt der App ergaben. Einige dieser Risiken finden sich auch im Prozessassessment wieder. Darunter fallen einige Punkte, die sich durch die zeitliche Limitierung ergaben, welche für eine markttaugliche App unabdingbar wären. Außerdem wurde festgestellt, dass das zu Anfang des Projekts besprochene Ausmaß der App nicht innerhalb des Projekts umgesetzt werden kann und nur eine vereinfachte Version dessen in die Tat umgesetzt werden sollte. Zu diesen Risiken entstanden zudem noch weitere Risiken, die Arbeitsweise der Teammitglieder betreffend, da diese unterschiedlich geübt in der entsprechenden Programmiersprache und App-Entwicklungstechnologie sind. Somit sollten einzelne Funktionen auch aus zeitlichen Gründen durch eine einfachere Alternative ersetzt werden, da eine aufwendigere Lösung den Zeitplan des Projekts zu

sehr gefährden könnte.

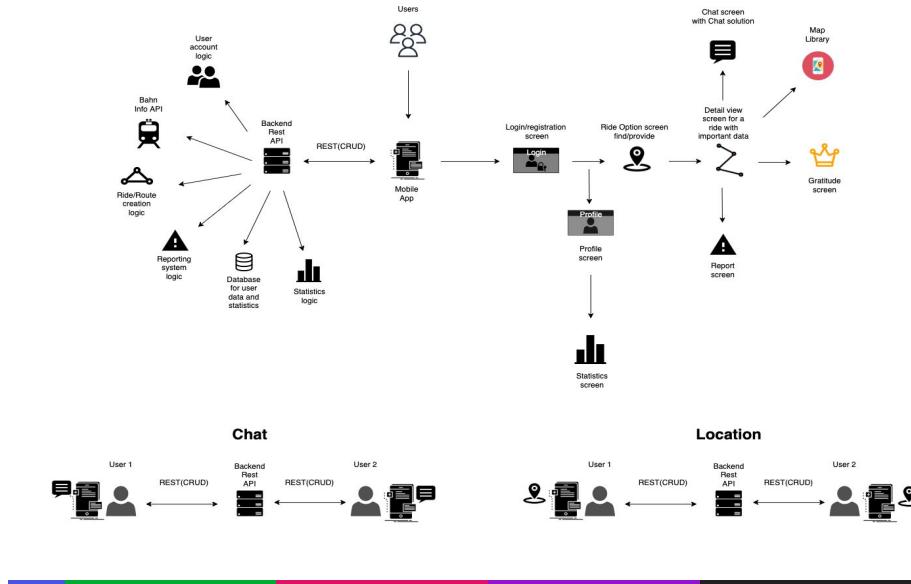
Risikoanalyse iteriert

Risiken	Bewertung	Maßnahmen
genug Nutzer erreichen	hoch	Abwägen, welche Entwicklungstechnologien genutzt werden
hohe Internetabhängigkeit	hoch	so wenig internetabhängige Funktionen integrieren wie möglich
Gültigkeit der Tickets verifizieren	mäßig	Verifizierung implementieren
viele mögliche Edge Cases	mäßig	Beschränkung auf einen hauptsächlichen Use Case
einige zeitaufwendige Aufgaben	hoch	Zeitliches Limit für alle Aufgaben festlegen
Aufgaben(wie der StreamChat) könnten sich als weitaus komplexer herausstellen	niedrig	Implementierung eines GiftedChats oder einer einfacheren Alternative



Auf dieser und der davorigen Folie sind noch einmal alle Risiken mit ihrer Bewertung und den Maßnahmen zu sehen. Auf dieser Folie finden sich auch die Ergänzungen, welche zuvor angesprochen wurden, wieder.

Architekturdiagramm iteriert



Innerhalb des Architekturdiagramms wurden nur noch Einzelheiten verändert beziehungsweise ergänzt. Das Diagramm selber, welches die Architektur der App als Ganzes zeigt wurde dabei nicht verändert, aus dem Grund, dass sich an der Architektur bis zum letztendlichen Main Prototype nichts verändert hatte und nur Funktionen und Features weggelassen wurden. Diese sollten im Architekturdiagramm allerdings nicht entfallen, da das Team der Ansicht war, durch das Architekturdiagramm lediglich einen Ausblick auf die umgesetzte, allerdings auch mögliche Architektur geben sollte. Zu der Chat Funktion, wessen Architektur separat, unterhalb des eigentlichen Architekturdiagramms demonstriert wurde, wurde ebenfalls die Architektur der Location visualisiert. Die Location wird somit, wie man der Abbildung entnehmen kann, durch Rest Abfragen von den Nutzern abgefragt und an das Backend der App weitergeleitet. Von da aus können sie weiterverarbeitet werden, sowie abgespeichert werden, um sie beispielsweise in der Karte abrufen zu können.

Prozessassessment

Was hat gut geklappt?

- Simulation einer Fahrt
- Implementation der ÖPNV Daten mit der HERE API
- jegliche Location abhängigen Funktionen
- Vernetzung zweier Nutzer über ihre Daten
- Zusammenarbeit im Team (Einteilung der Aufgaben je nach Stärken und Schwächen)
- Zielsetzung und Durchführung
- Zeitmanagement zum Einhalten der Fristen

Was hat nicht gut geklappt?

- Abdeckung einiger Edge Cases:
 - derzeit nur lineare Fahrten möglich
 - nur spontane Suche möglich
 - Verifizierung der Tickets
 - Bahn entfällt
- Use Case "Person mit Ticket dazu bringen, die App zu nutzen" wurde kaum beachtet
- Zahlungsmethode einführen
- Live Daten über KVB API beziehen
- Implementierung eines Stream Chats
- ungleiche Vorkenntnisse der Teammitglieder über Arbeit mit React Native und JS



Im Prozessassessment wurde über die gesamte Arbeit im Entwicklungsprojekt reflektiert und geschaut, wie diese, anhand von den zuvor gelieferten Artefakten und vor allem der Zielhierarchie, verlaufen ist. Dabei sollte zuerst eine Art Pro- und Contra Liste geführt werden, welche sich sowohl auf die Zielhierarchie aber auch auf die Projektrisiken beziehen sollte. Außerdem war es dem Team wichtig, auch auf interne Probleme oder Erfolge einzugehen, um auch dort eine Übersicht zu bekommen, was getan werden muss, um auch in diesem Kontext eine gelungene Zusammenarbeit zu garantieren. Abgesehen davon war ein großer Schwerpunkt der Liste, die erfolgreiche Durchführung eines ersten Use Case's. Hierbei ergaben sich einige Punkte, welche sich im Nachhinein als gut oder eben schlecht lösbar herausstellten und wie die jeweiligen Bewältigungsstrategien der PoC's letztendlich wahrgenommen werden konnten.

Prozessassessment

Was könnte man noch verbessern?

- Web-Socket statt Rest für Live Location Anfragen
- Web-Socket statt Rest für Chat
- weniger Flexibilität durch Abhängigkeit von Map Library
- bessere User Experience
- besseres Design
- bessere API für ÖPNV Daten
- Verifizierung der Tickets
- aktuellere Frontend Lösung für den Chat

Was könnte man noch dazu implementieren?

- Spenden Funktion
- Nutzer Profil
- Statistiken
- Melde Funktion
- andere Anmeldungsmöglichkeiten
- Semesterticket oder ähnliches sicher hochladen können
- Fahrtroute besser visualisieren
- geplante Fahrten
- komplexere Fahrten(bspw. mit Umsteigen)
- Dazusteigen zum Nutzer mit Ticket
- etc.



Zusätzlich zu der Umsetzung sollte noch eine Art Vision oder Aussicht auf weitere mögliche Schritte des Projekts in das Prozessassessment mit einfließen. Hierbei sollte allerdings unterschieden werden in Verbesserungsmöglichkeiten und komplett neue Features, die der App hinzugefügt werden könnten. Auch hier ist abermals erkennbar, dass aus zeitlichen Gründen einige Funktionen beziehungsweise Vorgänge durch einfachere Alternativen ersetzt werden mussten, die allerdings in Zukunft durch sicherere und praktischere Optionen verbessert werden könnten. So ergab sich als ein allumfassenderes Problem die Arbeit im Backend, da sie bis dato nur mit einer Rest Schnittstelle statt mit einem Web- Socket vollführt wurde, was für ein größeres Projekt eine geläufigere Option ist. Auch wurden hier einige Edge Case's genannt, welche zu Anfang des Projekts und auch im Laufe der Zeit gesammelt wurden und für eine komplexere App essentiell wären.

PoC: Jemanden finden, der die gleiche Strecke fährt durch GPS Tracking und Filter

Exit-Kriterien:

- Nutzer eigene Fahrt mittels Standort in App eingeben können über Karte
- Person mit übereinstimmender Fahrt finden
- Gemeinsame Fahrt vereinbaren

Fail-Kriterien:

- schlechte Internetverbindung
- gestörtes GPS Signal

Fallbacks:

- Standortinformationen sowie Fahrtinformationen werden durch die Eingabe der Nutzer erhalten und können so, wie geplant, weiterverarbeitet werden



Die Proof of Concept's, welche zuvor für den dritten Audit Termin entwickelt wurden, sollten noch einmal überarbeitet werden, da als Kritik geäußert wurde, dass mehr ein Use Case innerhalb der vom Team verfassten "PoC's" beschrieben wurde, als ein technisches Proof of Concept, welches sich auf einen spezifischen kritischen Pfad eines Use Case konzentriert. Dieser Kritik wurde nun entgegen gekommen und der Main Use Case, welcher vom Team umgesetzt werden sollte, wurde in mehrere PoC's unterteilt. Zum einen ergab sich so das PoC des GPS Trackings und anhand dessen die Filter- Funktion, da Nutzer über den Standort gefilterte Stationen auf der Karte angezeigt bekommen sollen, die mittels des Standorts und der Distanz zu den einzelnen Stationen gemessen werden sollen. Die Exit-, Fail- Kriterien, sowie Fallbacks unterscheiden sich nicht zu den zuvor formulierten Kriterien, sind allerdings nun aufgeteilt.

PoC: Implementierung der Live ÖPNV Daten in das System

Exit-Kriterien:

- die Daten können über die VRS und HERE API abgerufen werden und bei der Suche nach einem öffentlichen Verkehrsmittel in der Nähe und der Erstellung einer Fahrt erfolgreich implementiert und genutzt werden

Fail-Kriterien:

- kein Zugriff auf Live- Daten der ÖPNV
- schlechte Internetverbindung

Fallbacks:

- Informationen der ÖPNV manuell als kompaktere Version eingeben
- unvollständige Datensätze können durch Algorithmen angepasst werden
- Live Daten werden durch die Eingabe der Nutzer ersetzt



In dem zweiten iterierten Proof of Concept geht es einzig um die Implementierung der Live ÖPNV Daten in die App. Da, wie sich schon im Prozess des Rapid Prototypes herausstellte, die Beschaffung dieser Daten eine zeitliche sowie technisch umsetzbare Problematik darstellen würde, war es hier besonders wichtig, effiziente Fallbacks zu entwickeln, da die Fail- Kriterien, dass es gar nicht möglich sein würde, eine geeignete API zu finden, welche den Ansprüchen des Teams und der App entsprechen würde. Aus diesem Grund wurden mehrere Fallbacks formuliert, welche garantieren, dass diese Funktion in die Tat umgesetzt werden kann.

Code - PoC: Jemanden finden, der die gleiche Strecke fährt (Haltestellen Suchen Code)

```
const getDeparturesInLocation = async (req, res) => {
  const {location, radius} = req.query
  const url = `${urls.hereDepartures}&in=${location};r=${radius}`

  const formatDepartures = (departures) => departures.map(departure => {
    return {
      name: departure.transport.name,
      direction: departure.transport.headsign,
      time: moment(departure.time).format( format: "HH:mm"),
      delay: departure.delay
    }
  })

  try {
    const response = await axios.get(url)
    const result = response.data.boards.map(item => {
      return {
        name: item.place.name,
        start_station_cords: {
          latitude: item.place.location.lat,
          longitude: item.place.location.lng,
        },
        departures: formatDepartures(item.departures)
      }
    })

    return res.status(200).json({result: result})
  } catch (err) {
    console.error(err)
  }
}
```

Die zuvor nur als Mock-up demonstrierten Live Daten der öffentlichen Verkehrsmittel wurden nun, mittels der HERE API, in die App integriert. Mit Hilfe dieser können die Abfahrtszeiten der einzelnen Linien abgerufen werden, sowie die genaue GeoLocation jeder Haltestelle. Geplant war es diese Informationen von einer der lokalen ÖPNV Gewerbe zu bekommen, allerdings konnten diese Daten kaum beschaffen werden und nur in Formaten, die dem Team nicht geläufig waren und zu viel Zeit in Anspruch nahmen. Die erlangten Informationen sollten nun, wie im obigen Ausschnitt zu sehen, als Objekt abgerufen werden, um anschließend die gesamten Abfahrten an den Haltestellen in einem bestimmten Radius des Nutzers anzeigen lassen zu können.

Code - PoC: Jemanden finden, der die gleiche Strecke fährt (Fahrt Erstellen)

```
const createRide = ((req, res) => {
  const newRide = new Ride({...req.body})
  newRide.save()
    .then((ride) => {
      return res.status(200).json({result: ride});
    })
    .catch(err => {
      return console.log(err)
    });
})

const handleButtonPress = async (isCreator, rideData) => {
  if (isCreator) {
    await handlePost(urls.ride, rideData, setRide)
  } else {
    await handlePut( url: urls.ride + selectedElement._id, rideData, setRide)
  }
  setSelectedElement(null)
}
```

Wenn eine Fahrt von der Person mit Ticket erstellt wird, sollte sichergestellt werden, dass die Informationen für die Personen ohne Ticket in der Nähe sichtbar gemacht werden. Hierfür wurde sich dazu entschieden, die Daten der erstellten Fahrt vorerst in der Datenbank abzuspeichern, damit sie später von anderen Devices aus abgerufen werden kann. Dieser Vorgang ist in den obigen Code Snippets abgebildet, da er einen fundamentalen Vorgang für die Verbindung zweier Personen darstellt. Die Kommunikation mit der Datenbank erfolgt über REST und die Daten werden über CRUD Methoden angesprochen.

Die Erstellung einer Fahrt erfolgt nun nach Betätigen der Auswahl einer Haltestelle und der Auswahl einer Linie und deren Fahrtrichtung, sowie der geplanten Abfahrtszeit.

Code - PoC: Jemanden finden, der die gleiche Strecke fährt (Fahrten Suchen Code)

```
const getRidesByLocation = ((req, res) => {
  const {radius, location} = req.query
  Ride.find( {filter: {ride_status: "Created"}, callback: (err : CallbackError , rides : any[]) =>{
    const result = rides.filter(ride => getHaversineDistanceM(JSON.parse(location),ride.start_station_cords) < radius)

    return res.status(200).json({result: result})
  }}
})

const getHaversineDistanceM = (loc1, loc2) => {
  const RADIUS_OF_EARTH_IN_KM = 6371;
  const toRadian = angle => (Math.PI / 180) * angle;
  const distance = (a, b) => (Math.PI / 180) * (a - b);

  let [lat1, lon1] = [loc1.latitude, loc1.longitude]
  let [lat2, lon2] = [loc2.latitude, loc2.longitude]

  const disLat = distance(lat2, lat1);
  const disLon = distance(lon2, lon1);

  const a = Math.pow(Math.sin( x: disLat / 2), y: 2) + Math.pow(Math.sin( x: disLon / 2), y: 2)
    * Math.cos(toRadian(lat1)) * Math.cos(toRadian(lat2));
  const c = 2 * Math.asin(Math.sqrt(a));

  return (RADIUS_OF_EARTH_IN_KM * c) * 1000;
};
```

Die zuvor bereits als Mockup angedeuteten Funktionen, um eine bereits erstellte Fahrt im Umfeld des Nutzers ohne Ticket anzeigen lassen zu können, wurden nun lauffähig umgesetzt. In den oberen beiden Funktionen sieht man, wie die Funktionen aussehen, die letztendlich dazu führen, dass dem Nutzer eben nur Fahrten angezeigt werden, welche sich im unmittelbaren Radius der Person ohne Ticket befinden. Dies gilt, wie zuvor erklärt, dazu, dass die Funktionalität des Fahrten Suchens nur auf eine kurze Zeitspanne beschränkt ist, heißt, dass die Nutzer möglichst spontan nach einer passenden Verbindung suchen, sowie "eine Fahrt erstellen" würden. Dies hat den Vorteil, dass die App einen dynamischeren Ablauf und Verwendung bekommt die meisten Fahrten auch von Personen mit Ticket tendenziell eher kurz vor einer Fahrt eingegeben werden und Personen ohne Ticket somit eine höhere Chance haben, eine geeignete Mitfahrgelegenheit zu finden. In der oberen Funktion "getRidesByLocation" ist zu erkennen, welche Filter in diesem Vorgang greifen, um letztendlich die erstellten Fahrten aus der Datenbank zu bekommen, die den Anforderungen entsprechen. Dabei ist zuerst einmal wichtig, dass eine Fahrt, wie sie hier der Einfachheit halber genannt wird, von einem Nutzer mit Ticket erstellt wird. Wurde

dies getan, wird sie der Datenbank hinzugefügt und der Status steht erstmals auf "Created". Der Status ändert sich entsprechend, wenn Interaktionen mit dieser Fahrt getätigt werden. Wird beispielsweise eine Anfrage zur Mitfahrt eines anderen Nutzers gestellt, ändert sich dieser Status auf "Pending". Zusammen mit dem Status einer Fahrt sollte, wie schon gesagt, nur Fahrten angezeigt werden, welche sich in einem bestimmten Radius befinden. Um diesen Radius zu bestimmen und um zu schauen ob eine erstellte Fahrt sich innerhalb dieses Radius befindet, dient eine weitere Funktion, die "getHaversineDistanceM". Diese Funktion erhält die Koordinaten des aktuellen Nutzers und des jeweils anderen und rechnet vorerst die Entfernung der beiden Koordinaten aus und gibt diese Entfernung zurück. Anschließend wird innerhalb der Funktion "getRidesByLocation" geschaut, dass die Distanz einen kleineren Wert hat, als der festgelegte Radius, um im Anschluss auf der Karte angezeigt werden zu können.

Code - PoC: Jemanden finden, der die gleiche Strecke fährt (Fahrt Aktualisieren)

```
const updateRide = async (rideData) => {  
  await axios.put('http://localhost:8001/TicketFor2/ride/' + ride._id, rideData)  
    .then((res) => {  
      const newRide = res.data  
      setRide(newRide)  
      setShowConfirmation(false)  
    })  
    .catch((err) => {  
      console.log(err);  
    });  
};
```

```
const updateRide = ((req, res) => {  
  Ride.findByIdAndUpdate(req.params.id, {...req.body}, {new: true},  
    (err, result) => {  
      if(err) res.send(err)  
      else res.send(result)  
    })  
})
```

Um sowohl der Person mit Ticket, als auch der Person ohne Ticket, die einzelnen Vorgänge sichtbar zu machen, also dass eine Fahrt erstellt von der Person mit Ticket erstellt wurde, eine Mitfahrgelegenheit von der Person angefragt wurde, diese dann wiederum von der Person mit Ticket bestätigt wurde und die Fahrt abschließend abgeschlossen oder vorzeitig abgebrochen wurde (Created -> Pending -> Started -> Completed/Canceled), müssen auch diese Informationen ständig an das Backend übergeben werden, damit sie vom jeweils anderen Client abgerufen werden können. Auch hierfür wurde mit Updates an die Datenbank gearbeitet, um diese der anderen Person auf einfachem Wege vermitteln zu können und eine bessere Performance zu garantieren.

Da auch diese Funktion bereits beim Rapid Prototype funktionsfähig war, musste hier kaum noch etwas verändert werden. Trotzdem ist dies eine wichtige Funktion, die für den Ablauf einer Fahrt unabdingbar ist und sollte somit noch einmal aufgeführt werden.

Code - PoC: Wie sich die Personen in der Bahn/ im Bus finden

```
const [messages, setMessages] = useState( initialState: []);

export const handleMessageSend = async (ride, messages : any[] = [], callback) => {
  callback(prev => GiftedChat.append(prev, messages[0]))
  await handlePut( url: urls.chat + ride._id, messages[0], callback: () => null)
}

<GiftedChat
  messages={messages}
  onSend={messages => handleMessageSend(ride, messages, setMessages)}
  user={{
    _id: user._id,
    name: user.name,
    avatar: 'https://placeimg.com/140/140/any',
  }}
  showUserAvatar
  showAvatarForEveryMessage
  renderLoading={() => <View><Text>Loading...</Text></View>}
/>
```

Für den Main Prototype war ursprünglich die Implementierung eines StreamChats geplant. Dieser versprach ein eigenes Backend mitzubringen, so dass dieses nicht mehr hätte implementiert werden müssen. Allerdings gestaltete die Implementierung des StreamChats sich aufwendiger als erwartet, so dass das einlesen in die Materie und Sammeln der nötigen Komponenten einiges an Zeit raubte. Da nicht mit so einem hohen Zeitaufwand für die Implementierung gerechnet wurde und die Erledigung dessen daher erst spät angesetzt wurde, geriet das Team unter Zeitdruck und musste nach Alternativen suchen, welche nicht so einen hohen Zeitaufwand benötigten. Darum wurde auch beim Main Prototype auf den bereits vorher angedeuteten Gifted Chat gesetzt, welcher lediglich im Backend noch implementiert und im Frontend nur noch an die benötigten Komponenten angepasst werden musste.

Code - PoC: Wie endet eine Fahrt (Code)

```
const endRideCallback = () => {
  navigation.reset( state: {
    index: 0,
    routes: [{name: 'PostRide'}]
  })
}

const updateRideCallback = (res) => {
  const newRide = res
  setRide(newRide)
  if (newRide.ride_status === "Completed") {
    handlePut( url: urls.ride + ride._id, data: {ride_status: "Completed"}, endRideCallback)
  }
}

const updateOtherUserCallback = (res) => {
  const key = user._id === ride.ride_giver._id ? "ride_taker" : "ride_giver"
  setOtherUser(res[key])

  const usersSeparated = ride.ride_status === "Progress" && getHaversineDistanceM(res.ride_giver.location, res.ride_taker.location) > 100
  if (usersSeparated) {
    handlePut( url: urls.ride + ride._id, data: {ride_status: "Completed"}, updateRideCallback)
  }
}
```

Für das Beenden einer Fahrt konnten letztendlich alle Exit-Kriterien erfüllt werden. So besteht nun die Möglichkeit eine Fahrt manuell zu beenden oder aber diese beendet sich automatisch, aufgrund der Distanz, die zwischen den beiden Nutzern, welche eine Fahrt eingegangen sind, liegt. Hierfür wird ebenfalls auf die Funktion "getHaversineDistanceM" zurückgegriffen, welche einem abermals die Distanz zwischen den beiden Nutzern ausrechnet. Entfernen sich die Nutzer also weiter als 100 Meter wird der Status der Fahrt auf "Completed" geändert und die Nutzer werden auf den PostRide Screen geleitet. Dies garantiert dem Nutzer eine bessere User Experience, da er nicht nach jeder Fahrt extra in der App die Fahrt beenden muss, sondern diese, sobald sich ein Nutzer entfernt, und zum Beispiel aus dem Bus oder der Bahn aussteigt, von selbst beendet nachdem die Nutzer sich voneinander entfernen.

Fazit

- Planung der technischen Umsetzung früh genug starten
- Stärken und Schwächen der Gruppenmitglieder nutzen, um eine gute Aufgabenteilung zu garantieren
- Zeitfenster genauer festlegen, um sich nicht zu lange an einer Aufgabe aufzuhalten
- genauere Recherche der Ressourcen(mögliche API's, etc.) anstellen
- insgesamt guter Verlauf des Projekts
 - Ziele wurden erreicht
 - mit Veränderungen konnte gut umgegangen werden
 - Zeit konnte effizient genutzt werden



Letztendlich wurde noch ein Fazit, zusätzlich zum Prozessassessment, über das gesamte Projekt verfasst, in welchem auf die persönlichen Schlussfolgerungen eingegangen wurde und geschaut wurde, welche Aspekte man in einem zukünftigen Projekt verbessern könnte und welche Aspekte gut liefen. Dabei wurden einige Punkte aufgegriffen, die sich bereits durch vorherige Artefakte herauslesen konnten und welche das Team zu dem finalen Ergebnis geführt haben. Der Prozess war hierbei ein großer Bestandteil des Fazits. Zusammenfassend kann man also feststellen, dass bis auf ein paar Punkte, eine gute Zusammenarbeit geleistet wurde, bei welchem das Team mit seinem Vorhaben letzten endes einen funktionsfähigen Main Prototype vorzeigen konnte.

Projektplan

4. Audit			
Main Prototype	15.02.2022	50%	50%
Architekturdiagramm iteration	14.02.2022	30%	70%
Proof of concepts iteration	14.02.2022	60%	40%
Projektplan iteration	15.02.2022	30%	70%
Prozessassessment	16.02.2022	60%	40%



Für den 4. Audit wurden die nötigen Modelle noch einmal iteriert und der Main Prototype wurde fertiggestellt. Da sich die Architektur und die Risiken des Projekts ständig veränderte, durch neue Entscheidungen, die getroffen wurden, sollte auch das Architekturdiagramm stets überarbeitet werden. Außerdem wurden Kritikpunkte des letzten Audits aufgegriffen und die entsprechenden Artefakte wurden überarbeitet. Da der Main Prototype die meiste Zeit in Anspruch nehmen würde, wurde für ihn auch die meiste Zeit eingeplant.